

LEAF-Cloud: Layered Eco-centric Analytical Framework for Cloud

Prakhar Singhal
IIITH

February 18, 2025

Abstract

This document presents **LEAF-Cloud** (Layered Eco-centric Analytical Framework for Cloud), a comprehensive layered model for representing and analyzing Google Cloud Platform (GCP) infrastructure with an eco-centric perspective. The model employs colored Petri nets (CPNs) to simulate interactions among GCP services and estimate key operational parameters including latency, energy usage, and carbon footprint. In addition, LEAF-Cloud incorporates the dynamic scaling behavior of Google Kubernetes Engine (GKE) pods, whose runtime instances are estimated based on input workload, geolocation, and other factors. Detailed decision choices, explicit mathematical formulations, and integration with automation tools such as Terraform are provided.

1 Introduction

Cloud computing platforms such as Google Cloud Platform (GCP) offer a wide range of services to meet diverse workload demands. With the increasing emphasis on sustainability, it is critical to model not only performance but also the energy consumption and carbon footprint of cloud infrastructures. **LEAF-Cloud** is designed to address these challenges by using a layered, formal modeling approach based on colored Petri nets (CPNs). In addition to estimating latency, energy, and carbon metrics, LEAF-Cloud captures the dynamic scaling of GKE pods, allowing for workload-driven adjustments. This document details the design decisions, explicit mathematical formulations, and integration with automation tools such as Terraform.

2 Model Overview

LEAF-Cloud divides the modeling process into three main layers:

1. **Layer 1 – Abstract GCP Resource Types:** Aggregates broad resource buckets (Compute, Storage, Network, Security) that capture overall capacities, energy weights, and latency factors.
2. **Layer 2 – Specialized GCP Components:** Refines the abstract buckets into specific services (e.g., Compute Engine, GKE, Cloud Functions, Cloud Storage, Persistent Disks, VPC, and Cloud Load Balancers) with detailed attributes.
3. **Workload Layer:** Models applications as microservices or functions, characterized by request rates and resource demand vectors.

A simulation engine based on colored Petri nets is used to mimic the flow of tokens (representing workload jobs and resource utilization) through these layers. The simulation produces estimates of:

- **Latency** — total delay across resource transitions,
- **Energy Usage** — calculated via resource-specific energy functions,
- **Carbon Footprint** — derived by applying regional carbon factors to energy consumption.

3 Decision Choices

In developing LEAF-Cloud, several key decisions were made:

- **Use of Colored Petri Nets:** CPNs provide a formal method to model concurrency, resource sharing, and timing in complex systems. Their ability to attach attributes (colors) to tokens enables a fine-grained simulation of GCP workloads.
- **Layered Abstraction:** Dividing the model into abstract and specialized layers improves clarity and modularity. This stratified approach allows the model to be both deployment-agnostic and easily adaptable to different cloud providers.
- **Incorporating Dynamic Scaling:** GKE pods scale dynamically based on workload demand. Modeling scaling as a function of request rate, pod capacity, and geolocation ensures that the framework reflects real-world operational behavior.
- **Mapping to Terraform:** By mapping model outputs to Terraform configuration parameters, the approach facilitates integration with infrastructure automation and continuous monitoring frameworks.

- **Empirical Calibration:** Parameters such as energy weights, latency factors, and carbon factors are calibrated based on empirical data available from GCP documentation, third-party reports (e.g., [1]), and related academic work (e.g., [2], [3]).

4 Mathematical Model Description

The mathematical component of LEAF-Cloud estimates key performance and sustainability parameters by mapping token flows in the Petri net to quantifiable metrics. Each equation below is explicitly defined along with the meaning of its parameters.

4.1 Latency Estimation

For each transition t in the CPN, a delay $\delta(t)$ is associated. This delay may include factors such as processing time, communication latency, and waiting time due to resource contention. The overall latency L_{total} along a critical path is given by:

$$L_{\text{total}} = \sum_{t \in \text{CriticalPath}} \delta(t) + \Delta_{\text{congestion}}, \quad (1)$$

where:

- $\delta(t)$ represents the fixed delay of transition t (e.g., VM boot time or container startup time).
- $\Delta_{\text{congestion}}$ accounts for additional delays due to resource saturation or network congestion.

4.2 Energy Usage Estimation

The energy consumption for each resource i is modeled by the function:

$$E_i = w_i \cdot f_i(x_i), \quad (2)$$

where:

- w_i is the energy weight (or efficiency multiplier) for resource i .
- x_i denotes the resource utilization level (e.g., CPU, I/O).
- $f_i(x_i)$ is a mapping function (often linear) that translates the utilization level x_i into energy consumption.

For example, for Compute Engine:

$$E_{\text{compute}} = w_{\text{vm}} (\alpha_{\text{vm}} \cdot u + \beta_{\text{vm}}), \quad (3)$$

with u representing CPU utilization, and $\alpha_{\text{vm}}, \beta_{\text{vm}}$ being empirically derived coefficients.

The total energy consumption is aggregated as:

$$E_{\text{total}} = \sum_{i \in \{\text{compute, storage, network, security}\}} E_i + E_{\text{base}}, \quad (4)$$

where E_{base} is the baseline (idle) energy consumption.

4.3 Carbon Footprint Estimation

The environmental impact is quantified by converting energy usage into CO₂ equivalents using regional carbon factors:

$$CF_i = CF(\text{region}_i) \times E_i, \quad (5)$$

and the overall carbon footprint is given by:

$$CF_{\text{total}} = \sum_i CF_i. \quad (6)$$

4.4 Dynamic Scaling of GKE Pods

The dynamic scaling model for GKE pods estimates the number of pods N_{pods} required to handle a given workload:

$$N_{\text{pods}} = \left\lceil \frac{\lambda}{g \cdot R_{\text{pod}}} \right\rceil, \quad (7)$$

where:

- λ is the incoming request rate.
- R_{pod} is the processing capacity of a single pod.
- g is a geolocation factor (with $g < 1$ typically indicating lower effective capacity due to network conditions).
- The ceiling function $\lceil \cdot \rceil$ ensures an integer number of pods.

5 GCP Resources and Services Modeling

LEAF-Cloud explicitly models the wide range of GCP services. For clarity, similar services are grouped and modeled together where appropriate:

5.1 1. Compute

- **Compute Engine (VMs):** Modeled as tokens with attributes (e.g., CPU, memory) in the CPN. Energy and latency functions are applied based on resource utilization.
- **Google Kubernetes Engine (GKE):** Each pod is modeled as a token with dynamic scaling captured by the equation in Section 3.4. Pods inherit similar energy and latency characteristics as VMs.
- **App Engine:** Treated as an abstract compute service with automatic scaling. Modeled similarly to Compute Engine but with built-in latency reduction factors.
- **Cloud Functions & Cloud Run:** Modeled as event-triggered, ephemeral compute instances. They are represented as short-lived tokens whose delays and energy consumption are functions of execution time and payload size.

5.2 2. Storage & Databases

- **Object & File Storage (Cloud Storage, Filestore):** Modeled as storage buckets with associated access latency and energy functions based on I/O operations.
- **Block Storage (Persistent Disk):** Modeled as storage resources with performance parameters (e.g., IOPS, throughput) and energy consumption tied to disk activity.
- **Relational Databases (Cloud SQL, Cloud Spanner):** Represented as compute resources with integrated storage. Query processing is modeled via transitions with latency and energy consumption proportional to the query load.
- **NoSQL & In-Memory Databases (Cloud Bigtable, Firestore/-Cloud Datastore, Memorystore):** Modeled as dynamic database services with low latency. In-memory databases (Memorystore) have energy models based on throughput and memory usage.

5.3 3. Networking & Content Delivery

- **Virtual Private Cloud (VPC):** Modeled as a networking layer that governs token flow between resources, with inherent latency and energy overhead.
- **Cloud Load Balancing:** Represented as transitions that distribute tokens (requests) across compute resources, with models for balancing delay.

- **Cloud CDN:** Modeled as a caching layer that reduces access latency; its effect is incorporated as a latency reduction factor.
- **Cloud DNS:** Modeled as a lookup service with minimal latency.
- **Cloud Interconnect & Network Service Tiers:** Modeled as dedicated network paths with defined bandwidth, latency, and energy consumption.
- **Cloud VPN:** Modeled as an encrypted channel introducing additional latency and energy overhead.

5.4 4. Big Data & Analytics

- **BigQuery:** Modeled as a query engine with latency functions dependent on query complexity and dataset size; energy usage is scaled with processing demand.
- **Dataflow & Dataproc:** Represented as processing pipelines or clusters. They are modeled similarly to compute resources with additional dynamic scaling for batch or streaming workloads.
- **Pub/Sub:** Modeled as a messaging queue, with delays reflecting message propagation and processing.
- **Data Fusion, Dataprep & Data Studio:** Treated as analytical layers with transformation and visualization tasks that contribute to overall latency and energy consumption.

5.5 5. Artificial Intelligence & Machine Learning

- **AI Platform & AutoML:** Modeled as compute-intensive services with high energy consumption. Separate training and prediction phases are modeled with distinct latency and energy functions.
- **API Services (Cloud Vision API, Cloud Speech-to-Text, Cloud Natural Language API, Cloud Translation API, Dialogflow, Recommendations AI):** Modeled as API endpoints where each request incurs a processing delay and energy cost based on computational complexity.

5.6 6. Identity, Security & Compliance

- **IAM & Cloud Identity:** Modeled as control layers with minimal latency, ensuring secure access control.
- **Security Services (Cloud Security Command Center, Cloud KMS, Cloud Armor, BeyondCorp Enterprise, Security Health Analytics, Data Loss Prevention API):** Modeled as middleware

layers that add security processing delays and additional energy consumption for encryption, decryption, and analysis.

5.7 7. Developer Tools & Application Integration

- **CI/CD and Integration Tools (Cloud SDK, Cloud Build, Container/Artifact Registry, Cloud Source Repositories, Cloud Code):** Modeled as auxiliary services with lower latency and energy profiles that support automated deployments.
- **Task Management (Cloud Tasks, Cloud Scheduler, Cloud Pub/Sub):** Modeled as orchestration services, with delays modeled for scheduling and message propagation.

5.8 8. Operations, Monitoring & Management

- **Management Interfaces (Cloud Console, Cloud Shell):** Represented as user-facing layers with minimal overhead.
- **Monitoring Tools (Cloud Monitoring, Cloud Logging, Cloud Trace, Cloud Debugger, Cloud Profiler, Deployment Manager):** Modeled as background services that contribute to system overhead by adding baseline energy and latency factors.

5.9 9. Internet of Things (IoT)

- **Cloud IoT Core:** Modeled as a gateway for IoT devices, where tokens represent device telemetry. Energy and latency functions are adjusted based on data volume and frequency.

5.10 10. Hybrid & Multi-cloud Solutions

- **Anthos & Migrate for Compute Engine:** Modeled as orchestration and migration layers that facilitate integration between on-premises and cloud environments. They introduce additional latency for data replication and migration while incurring extra energy consumption.

5.11 11. API Management & Integration

- **Apigee API Management & Cloud Endpoints:** Modeled as API gateways that manage external requests. These are represented with

additional security and latency overheads, and their energy models scale with request volume.

5.12 12. Enterprise & Productivity

- **Google Workspace & Cloud Search:** Modeled as end-user productivity and search services. While their energy and latency profiles are lower compared to compute-intensive services, they are integrated as essential layers within the overall simulation.

6 Mapping to Terraform for GCP

The outputs from the abstract and specialized models are integrated with GCP via Terraform. For instance, a GKE cluster configuration might be expressed as:

```

1 resource "google_container_cluster" "gke_cluster" {
2   name           = var.cluster_name
3   location       = var.zone
4   initial_node_count = var.initial_node_count
5
6   node_config {
7     machine_type = var.node_machine_type
8     labels = {
9       energy_weight = var.energy_weight_gke
10    }
11  }
12
13  autoscaling {
14    min_node_count = var.min_nodes
15    max_node_count = var.max_nodes
16  }
17 }
18
19 output "gke_scaling_estimate" {
20   value = "Estimated_Num_Pods"
21 }

```

Listing 1: Terraform configuration for a GKE cluster with autoscaling

The estimated number of pods is computed as:

$$\text{Estimated_Num_Pods} = \left\lceil \frac{\text{var.request_rate}}{\text{var.geolocation_factor} \cdot \text{var.pod_capacity}} \right\rceil,$$

with the variables `pod_capacity`, `request_rate`, and `geolocation_factor` derived from the LEAF-Cloud simulation outputs.

7 Conclusion and Future Work

This report has presented **LEAF-Cloud: Layered Eco-centric Analytical Framework for Cloud**, a layered approach to model GCP infrastructure and dynamically scale services (such as GKE pods). The framework employs colored Petri nets to simulate resource interactions and explicitly estimates key parameters—latency, energy usage, and carbon footprint—using detailed mathematical formulations and service-specific models.

Future work can focus on:

- Integrating real-time data from Cloud Monitoring to refine model parameters and workload definitions.
- Extending the simulation to cover multi-region workload distribution and different cloud provider integrations.
- Applying machine learning techniques for online parameter tuning to enhance prediction accuracy.

References

- [1] Rajkumar Buyya et al. “Cloud computing and emerging IT platforms: Vision, type, and reality for delivering computing as the 5th utility”. In: *Future Generation Computer Systems* 25.6 (2009), pp. 599–616.
- [2] YC Lee and AY Zomaya. “Energy efficient utilization of resources in cloud computing systems”. In: *Journal of Supercomputing* 60.2 (2012), pp. 268–280.
- [3] Muhammad Rizwan Ali et al. “Petri Net based modeling and analysis for improved resource utilization in cloud computing”. In: *PeerJ Computer Science* 7 (2021), e351.